DSA ASSIGNMENT -1

NAME – KAMINI MISHRA

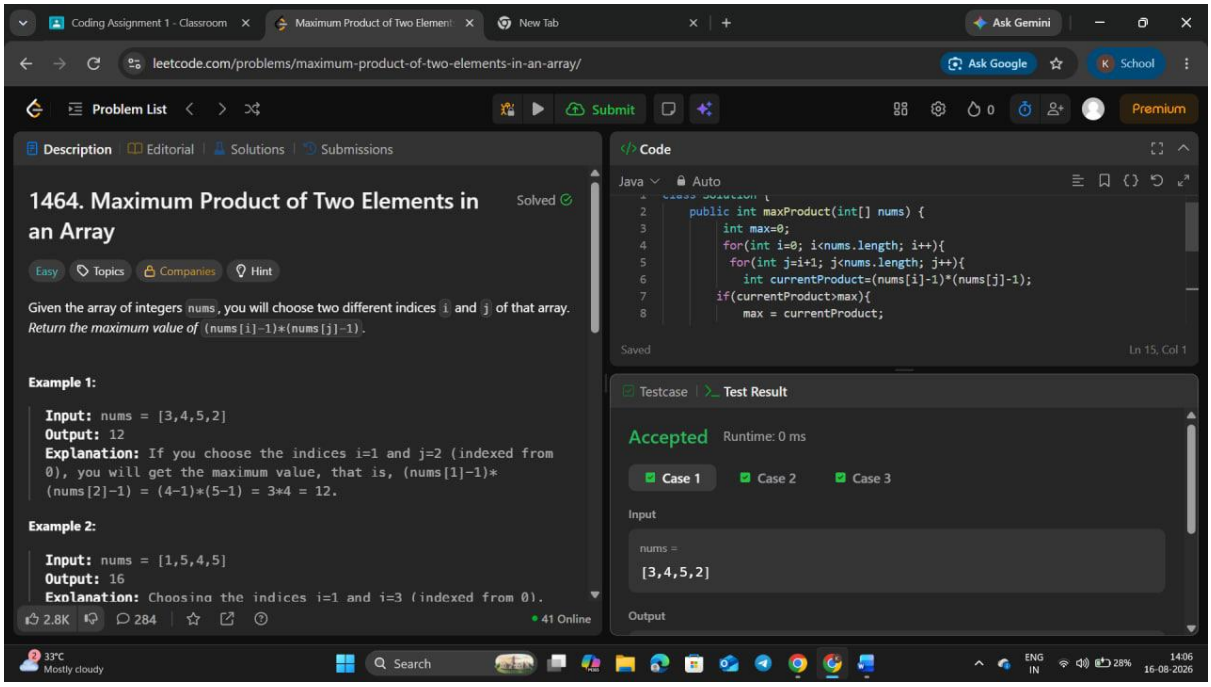
UNIVERSITY ROLL NO. – 12515000734

CLASS ROLL NO. – 25

SECTION – FB

1. 1464-Maximum Product of Two Elements in an Array

```
class Solution {
public int maxProduct(int[] nums) {
int max=0;
for(int i=0; i<nums.length; i++){
for(int j=i+1; j<nums.length; j++){
int currentProduct=(nums[i]-1)*(nums[j]-1);
if(currentProduct>max){
max = currentProduct;
}
}
}
return max;
}
}
```

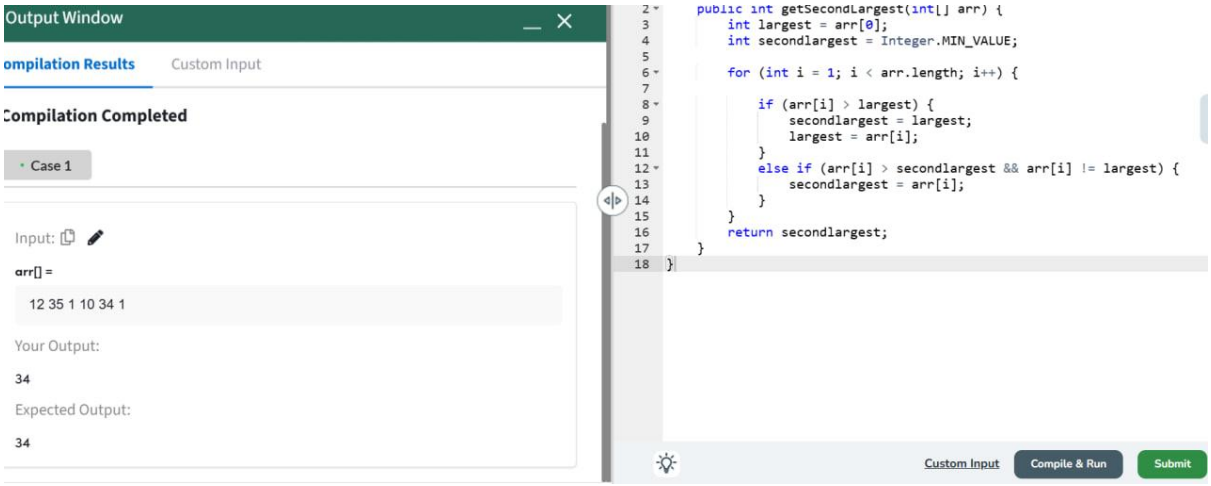


2. Second largest

```
class Solution {
public int getSecondLargest(int[] arr) {
int largest = arr[0];
int secondlargest = Integer.MIN_VALUE;

for (int i = 1; i < arr.length; i++) {

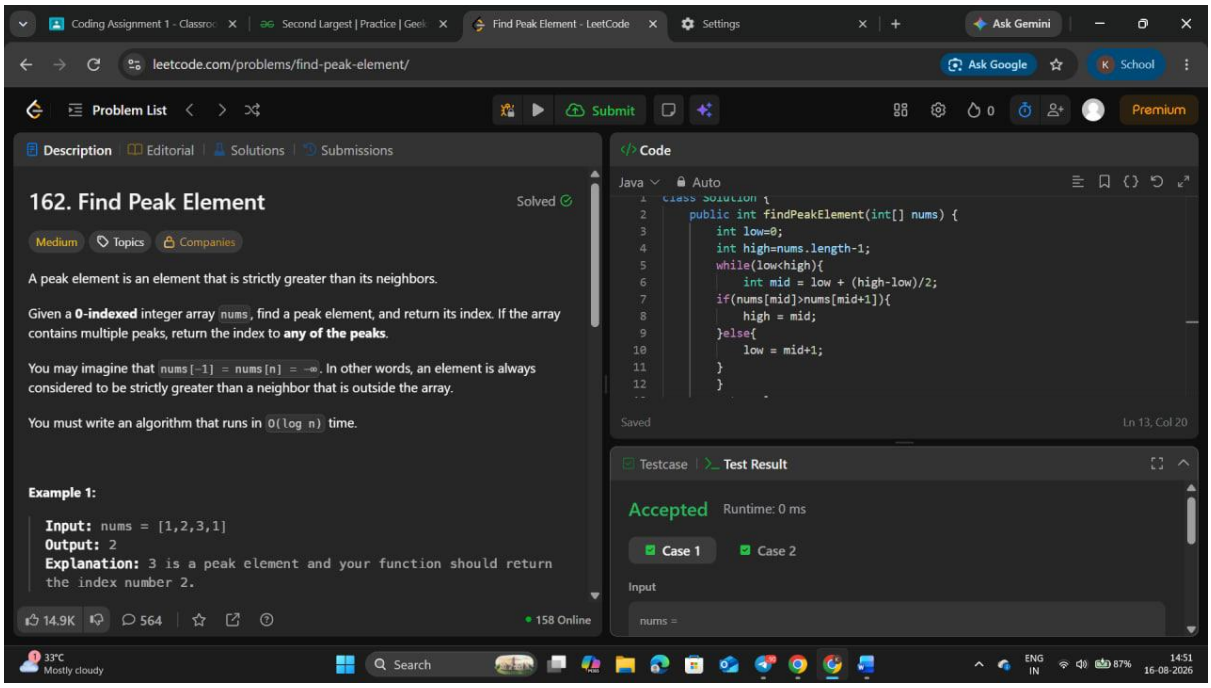
if (arr[i] > largest) {
secondlargest = largest;
largest = arr[i];
}
else if (arr[i] > secondlargest && arr[i] != largest) {
secondlargest = arr[i];
}
}
return secondlargest;
}
}
```



3.162- Find Peak Element

```
class Solution {
public int findPeakElement(int[] nums) {
int low=0;
int high=nums.length-1;
while(low<high){
int mid = low + (high-low)/2;
if(nums[mid]>nums[mid+1]){
high = mid;
}else{
low = mid+1;
```

}
}
return low;
}
}

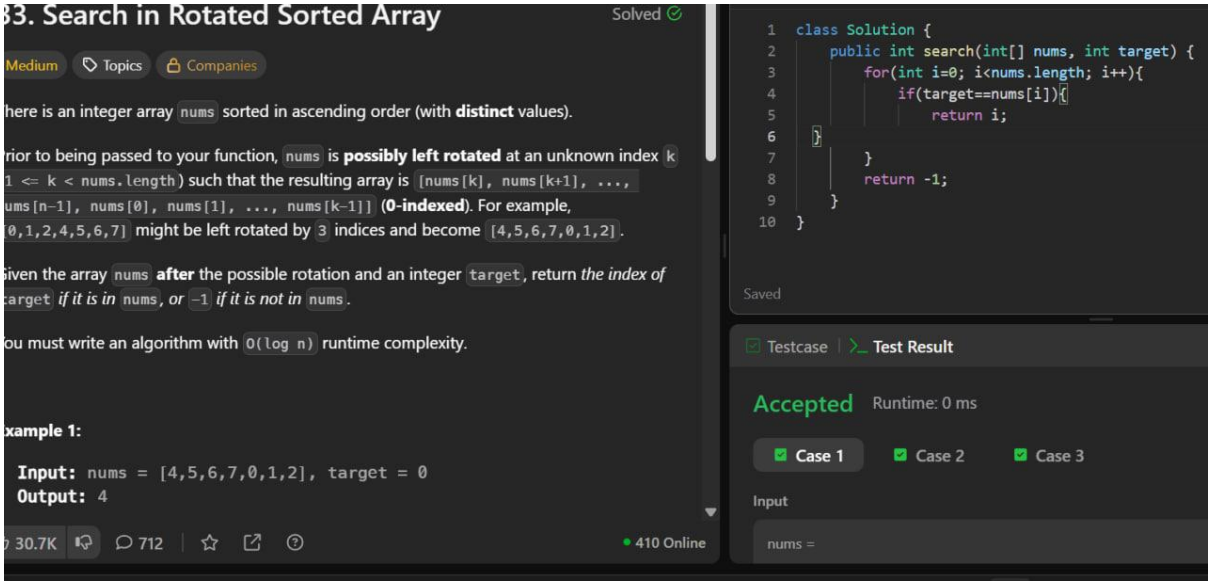


4. 33-Search in Rotated Sorted Array

class Solution {
public int search(int[] nums, int target) {
for(int i=0; i<nums.length; i++){
if(target==nums[i]){
return i;
}
}
return -1;

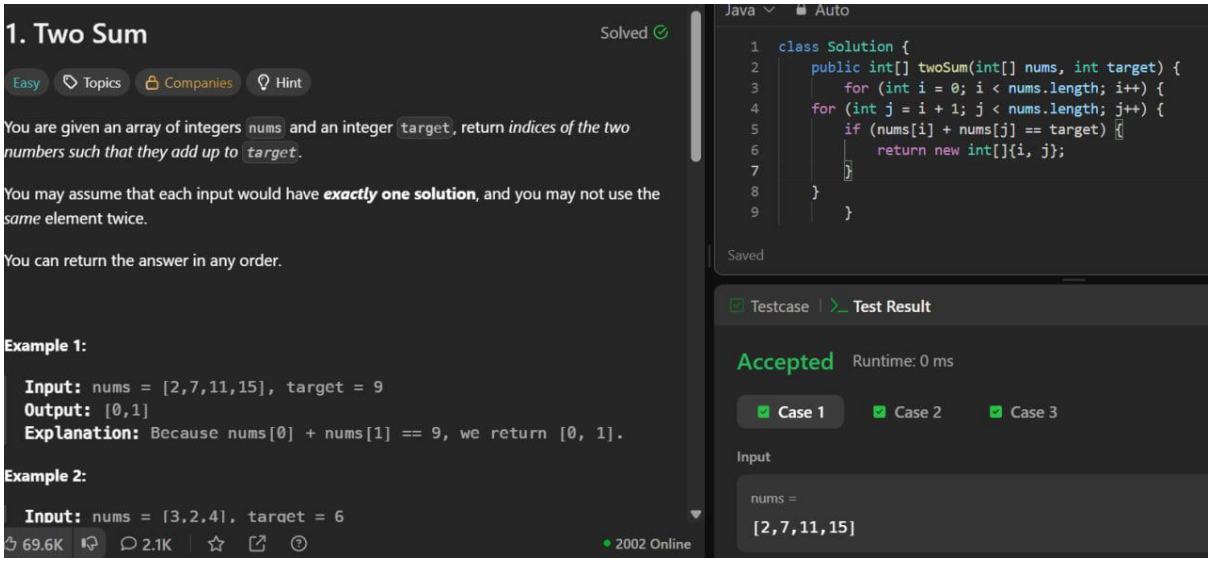
}

}



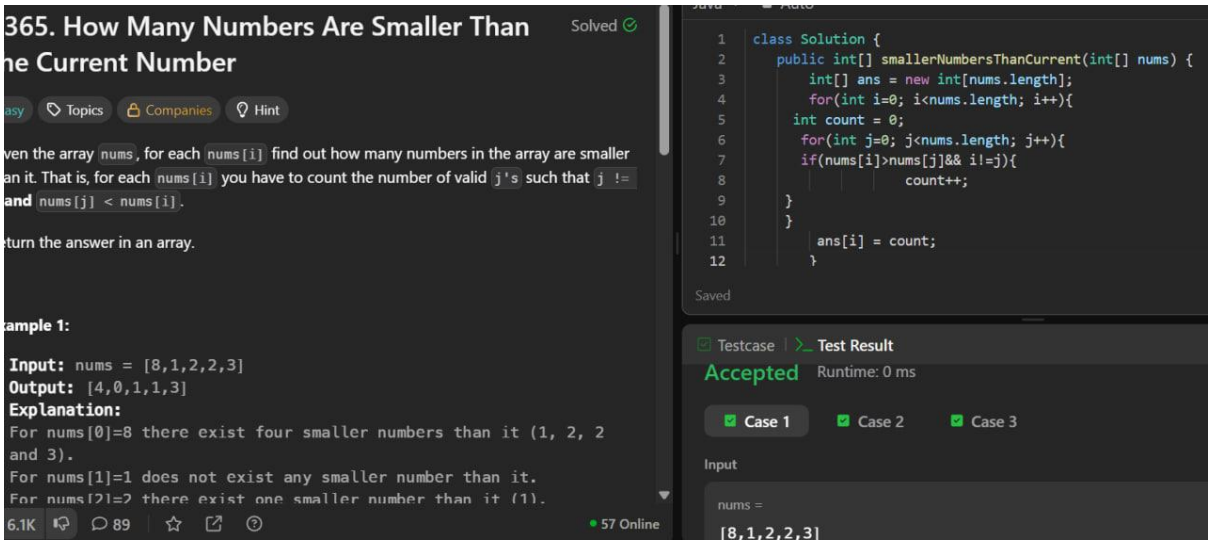
5. two sum

```
class Solution {
public int[] twoSum(int[] nums, int target) {for (int i = 0; i < nums.length; i++) {
for (int j = i + 1; j < nums.length; j++) {
if (nums[i] + nums[j] == target) {
return new int[]{i, j};
}
}
}
return new int[]{-1, -1};
}
}
```



6. 1365 How many Numbers are Smaller Than Current Number

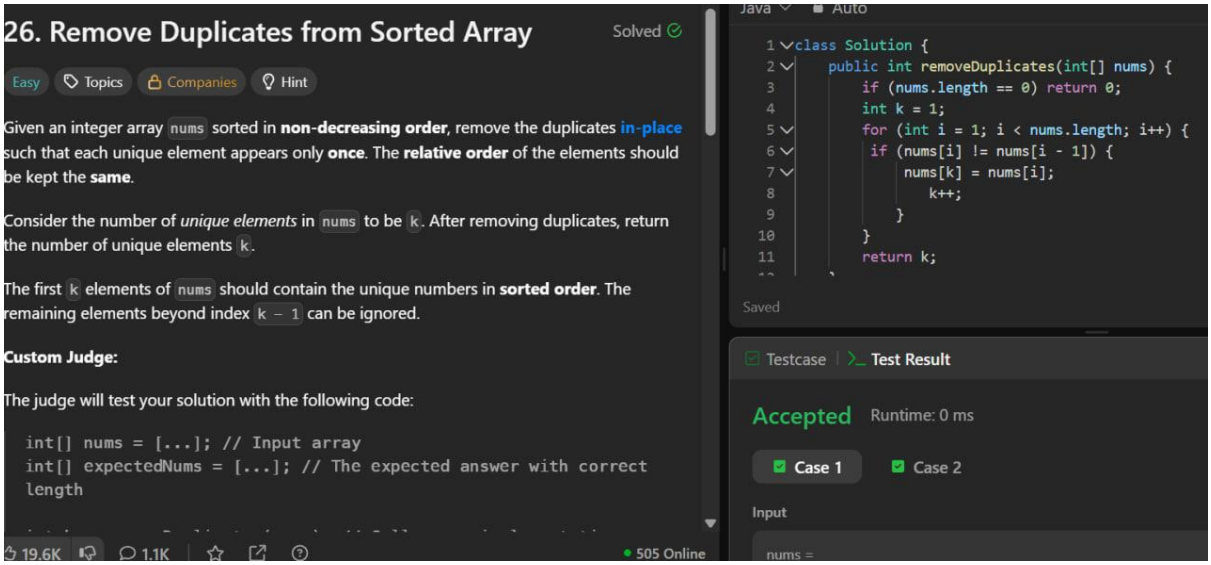
```
class Solution {
public int[] smallerNumbersThanCurrent(int[] nums) {
int[] ans = new int[nums.length];
for(int i=0; i<nums.length; i++){
int count = 0;
for(int j=0; j<nums.length; j++){
if(nums[i]>nums[j]&& i!=j){
count++;
}
}
ans[i] = count;
}
return ans;
}
}
```



7. 26 - Remove duplicates from sorted array

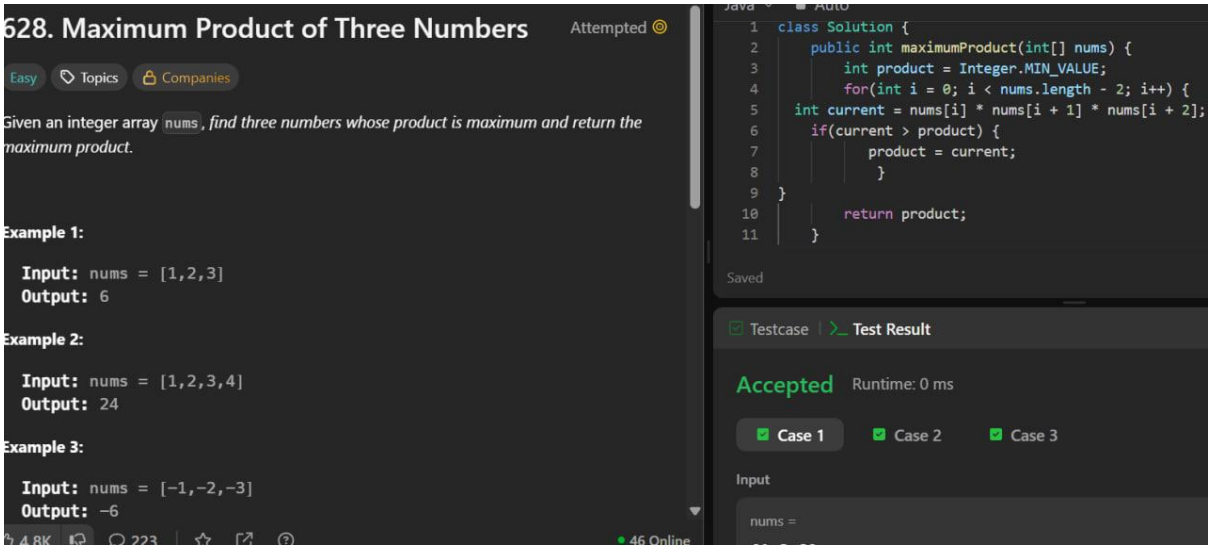
```
class Solution {
public int removeDuplicates(int[] nums) {
if (nums.length == 0) return 0;
int k = 1;
for (int i = 1; i < nums.length; i++) {
if (nums[i] != nums[i - 1]) {
nums[k] = nums[i];
k++;
}
}
return k;
}
```

}



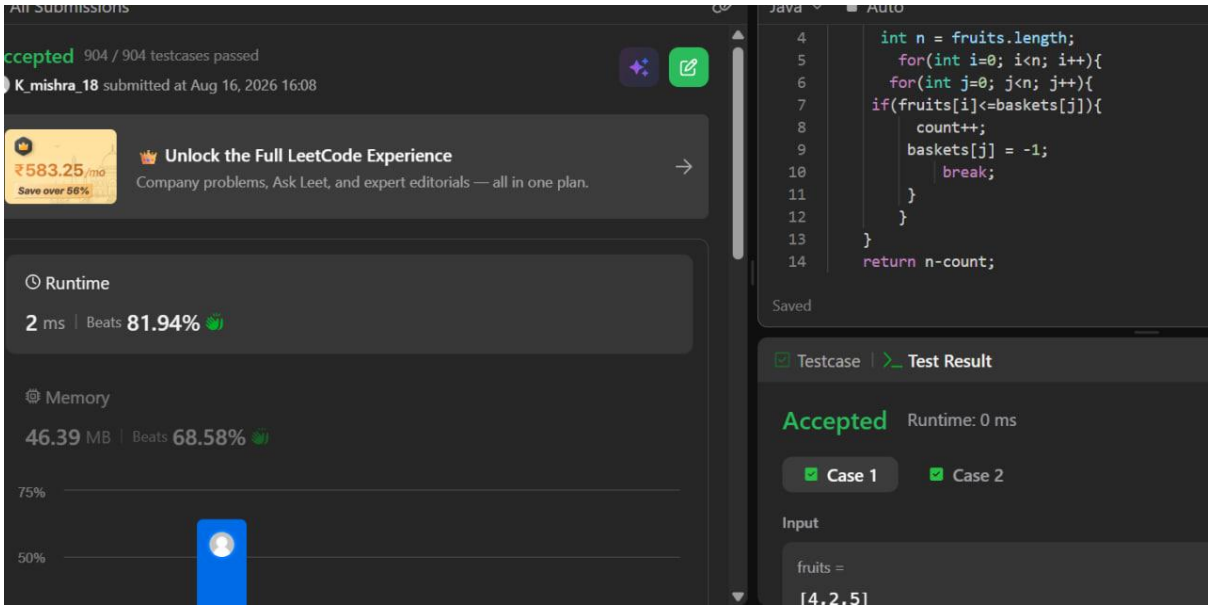
8. 628. Maximum product of three numbers

```
class Solution {
public int maximumProduct(int[] nums) {
int product = Integer.MIN_VALUE;
for(int i = 0; i < nums.length - 2; i++) {
int current = nums[i] * nums[i + 1] * nums[i + 2];
if(current > product) {
product = current;
}
}
return product;
}
}
```



9. 3477 . fruits into baskets II

```
class Solution {
public int numOfUnplacedFruits(int[] fruits, int[] baskets) {
int count =0;
int n = fruits.length;
for(int i=0; i<n; i++){
for(int j=0; j<n; j++){
if(fruits[i]<=baskets[j]){
count++;
baskets[j] = -1;
break;
}
}
}
return n-count;
}
}
```



10. 121-best time to buy and sell stock

```
class Solution {
public int maxProfit(int[] prices) {
int min = prices[0];
int profit = 0;
for (int i = 1; i < prices.length; i++) {
if (prices[i] < min) {
min = prices[i];
}
else if (prices[i] - min > profit) {
```

```
profit = prices[i] - min;
}
}
return profit;
}
}
```

